

LIkhaKahoy Woodcraft

Week 14 – Final Project Phase 5 | Optimization Report

During this lab activity, several Business Intelligence (BI) queries from the Woodcraft Shop database were tested and optimized to improve performance. The selected queries included Monthly Revenue, Top Customers by Spending, Best-Selling Products, Revenue by Category, and Daily Sales Performance. These queries were chosen because they use JOIN, GROUP BY, and ORDER BY operations, which may become slower when handling large amounts of data.

Before optimization, the queries were analyzed using the EXPLAIN command in MySQL Workbench. The initial execution times ranged from around 0.937 seconds to 0.032 second depending on the complexity of the query. Some queries scanned many rows and performed full table scans because indexes were not yet applied. Queries involving multiple table joins, especially the Top Customers and Best-Selling Products queries, required more processing time compared to simple SUM and AVG queries.

To improve performance, indexes were added to important columns used in JOIN and filtering operations. Indexes were created on `customer_id`, `product_id`, `order_id`, `category_id`, and `payment_date`. These indexes helped MySQL find matching records faster instead of checking every row in the table. The EXPLAIN results after optimization showed that indexes were being used properly during query execution. The queries were also reviewed to ensure proper JOIN usage and efficient aggregation.

After optimization, query execution became faster and more efficient. Some queries improved from around 0.234 second to approximately 0.00 seconds after indexing. The EXPLAIN output also showed better execution types such as `ref` and `index`, which means fewer rows were scanned during execution. Although the returned results remained the same, the database processed the queries more efficiently after optimization.

This activity showed the importance of indexing and query optimization in large-scale database systems. For future database development, indexes should be added to columns frequently used in JOIN, WHERE, GROUP BY, and ORDER BY clauses. Developers should also avoid unnecessary nested queries and avoid using `SELECT *` when only specific columns are needed. Regular use of EXPLAIN can help identify performance issues early and improve database scalability and efficiency.